3

Enhancements to the GROUP BY Clause

ORACLE

Copyright © Oracle Corporation, 2001. All rights reserved.

Objectives

After completing this lesson, you should be able to do the following:

- **Use the `ROLLUP` operation to produce subtotal values**
- **Use the `CUBE` operation to produce cross-tabulation values**
- **Use the `GROUPING` function to identify the row values created by `ROLLUP` or `CUBE`**
- **Use `GROUPING SETS` to produce a single result set**

ORACLE

3-2 Copyright © Oracle Corporation, 2001. All rights reserved.

Lesson Aim

In this lesson you learn how to:

- Group data for obtaining the following:
  - Subtotal values by using the `ROLLUP` operator
  - Cross-tabulation values by using the `CUBE` operator
- Use the `GROUPING` function to identify the level of aggregation in the results set produced by a `ROLLUP` or `CUBE` operator.
- Use `GROUPING SETS` to produce a single result set that is equivalent to a `UNION ALL` approach.

Review of Group Functions

Group functions operate on sets of rows to give one result per group.

```
SELECT        [column,] group_function(column). . .
FROM          table
[WHERE        condition]
[GROUP BY     group_by_expression]
[ORDER BY     column];
```

Example:

```
SELECT AVG(salary), STDDEV(salary),
       COUNT(commission_pct),MAX(hire_date)
FROM   employees
WHERE  job_id LIKE 'SA%';
```

ORACLE

3-3 Copyright © Oracle Corporation, 2001. All rights reserved.

Group Functions

You can use the `GROUP BY` clause to divide the rows in a table into groups. You can then use the group functions to return summary information for each group. Group functions can appear in select lists and in `ORDER BY` and `HAVING` clauses. The Oracle Server applies the group functions to each group of rows and returns a single result row for each group.

Types of Group Functions

Each of the group functions `AVG`, `SUM`, `MAX`, `MIN`, `COUNT`, `STDDEV`, and `VARIANCE` accept one argument. The functions `AVG`, `SUM`, `STDDEV`, and `VARIANCE` operate only on numeric values. `MAX` and `MIN` can operate on numeric, character, or date data values. `COUNT` returns the number of nonnull rows for the given expression. The example in the slide calculates the average salary, standard deviation on the salary, number of employees earning a commission and the maximum hire date for those employees whose `JOB_ID` begins with SA.

Guidelines for Using Group Functions

- The data types for the arguments can be `CHAR`, `VARCHAR2`, `NUMBER`, or `DATE`.
- All group functions except `COUNT(*)` ignore null values. To substitute a value for null values, use the `NVL` function. `COUNT` returns either a number or zero.
- The Oracle Server implicitly sorts the results set in ascending order of the grouping columns specified, when you use a `GROUP BY` clause. To override this default ordering, you can use `DESC` in an `ORDER BY` clause.

Review of the GROUP BY Clause

Syntax:

```
SELECT        [column,] group_function(column). . .
FROM          table
[WHERE        condition]
[GROUP BY     group_by_expression]
[ORDER BY     column];
```

Example:

```
SELECT   department_id, job_id, SUM(salary),
         COUNT(employee_id)
FROM     employees
GROUP BY department_id, job_id ;
```

ORACLE

Copyright © Oracle Corporation, 2001. All rights reserved.

Review of GROUP BY Clause

The example illustrated in the slide is evaluated by the Oracle Server as follows:

- The SELECT clause specifies that the following columns are to be retrieved:
  - Department ID and job ID columns from the EMPLOYEES table
  - The sum of all the salaries and the number of employees in each group that you have specified in the GROUP BY clause
- The GROUP BY clause specifies how the rows should be grouped in the table. The total salary and the number of employees are calculated for each job ID within each department. The rows are grouped by department ID and then grouped by job within each department.

DEPARTMENT_ID	JOB_ID	SUM(SALARY)	COUNT(EMPLOYEE_ID)
10	AD_ASST	4400	1
20	MK_MAN	13000	1
20	MK_REP	6000	1
50	ST_CLERK	11700	4
...			
110	AC_ACCOUNT	8300	1
110	AC_MGR	12000	1
	SA_REP	7000	1

13 rows selected.

Review of the HAVING Clause

```
SELECT        [column,] group_function(column)...
FROM          table
[WHERE        condition]
[GROUP BY     group_by_expression]
[HAVING       having_expression]
[ORDER BY     column];
```

- **Use the HAVING clause to specify which groups are to be displayed.**
- **You further restrict the groups on the basis of a limiting condition.**

ORACLE

3-5 Copyright © Oracle Corporation, 2001. All rights reserved.

The HAVING Clause

Groups are formed and group functions are calculated before the HAVING clause is applied to the groups. The HAVING clause can precede the GROUP BY clause, but it is recommended that you place the GROUP BY clause first because it is more logical.

The Oracle Server performs the following steps when you use the HAVING clause:

1. Groups rows
2. Applies the group functions to the groups and displays the groups that match the criteria in the HAVING clause

```
SELECT department_id, AVG(salary)
FROM   employees
GROUP  BY  department_id
HAVING AVG(salary) >9500;
```

DEPARTMENT_ID	AVG(SALARY)
80	10033.3333
90	19333.3333
110	10150

The example displays department ID and average salary for those departments whose average salary is greater than $9,500.

GROUP BY with ROLLUP and CUBE Operators

- Use ROLLUP or CUBE with GROUP BY to produce superaggregate rows by cross-referencing columns.
- ROLLUP grouping produces a results set containing the regular grouped rows and the subtotal values.
- CUBE grouping produces a results set containing the rows from ROLLUP and cross-tabulation rows.

ORACLE

3-6 Copyright © Oracle Corporation, 2001. All rights reserved.

GROUP BY with the ROLLUP and CUBE Operators

You specify ROLLUP and CUBE operators in the GROUP BY clause of a query. ROLLUP grouping produces a results set containing the regular grouped rows and subtotal rows. The CUBE operation in the GROUP BY clause groups the selected rows based on the values of all possible combinations of expressions in the specification and returns a single row of summary information for each group. You can use the CUBE operator to produce cross-tabulation rows.

Note: When working with ROLLUP and CUBE, make sure that the columns following the GROUP BY clause have meaningful, real-life relationships with each other; otherwise the operators return irrelevant information.

The ROLLUP and CUBE operators are available only in Oracle8*i* and later releases.

ROLLUP Operator

```
SELECT        [column,] group_function(column). . .
FROM          table
[WHERE        condition]
[GROUP BY     [ROLLUP] group_by_expression]
[HAVING       having_expression];
[ORDER BY     column];
```

- **ROLLUP is an extension to the GROUP BY clause.**
- **Use the ROLLUP operation to produce cumulative aggregates, such as subtotals.**

ORACLE

3-7

Copyright © Oracle Corporation, 2001. All rights reserved.

The ROLLUP Operator

The ROLLUP operator delivers aggregates and superaggregates for expressions within a GROUP BY statement. The ROLLUP operator can be used by report writers to extract statistics and summary information from results sets. The cumulative aggregates can be used in reports, charts, and graphs.

The ROLLUP operator creates groupings by moving in one direction, from right to left, along the list of columns specified in the GROUP BY clause. It then applies the aggregate function to these groupings.

Note: To produce subtotals in *n* dimensions (that is, *n* columns in the GROUP BY clause) without a ROLLUP operator, *n*+1 SELECT statements must be linked with UNION ALL. This makes the query execution inefficient, because each of the SELECT statements causes table access. The ROLLUP operator gathers its results with just one table access. The ROLLUP operator is useful if there are many columns involved in producing the subtotals.

ROLLUP Operator Example

```
SELECT   department_id, job_id, SUM(salary)
FROM     employees
WHERE    department_id < 60
GROUP BY ROLLUP(department_id, job_id);
```

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
10		4400
20	MK_MAN	13000
20	MK_REP	6000
20		19000
50	ST_CLERK	11700
50	ST_MAN	5800
50		17500
		40900

9 rows selected.

1 2 3

ORACLE

Copyright © Oracle Corporation, 2001. All rights reserved.

Example of a ROLLUP Operator

In the example in the slide:

- Total salaries for every job ID within a department for those departments whose department ID is less than 60 are displayed by the `GROUP BY` clause (labeled 1)
- The `ROLLUP` operator displays:
  - Total salary for those departments whose department ID is less than 60 (labeled 2)
  - Total salary for all departments whose department ID is less than 60, irrespective of the job IDs (labeled 3)
- All rows indicated as 1 are regular rows and all rows indicated as 2 and 3 are superaggregate rows.

The `ROLLUP` operator creates subtotals that roll up from the most detailed level to a grand total, following the grouping list specified in the `GROUP BY` clause. First it calculates the standard aggregate values for the groups specified in the `GROUP BY` clause (in the example, the sum of salaries grouped on each job within a department). Then it creates progressively higher-level subtotals, moving from right to left through the list of grouping columns. (In the preceding example, the sum of salaries for each department is calculated, followed by the sum of salaries for all departments.)

- Given n expressions in the `ROLLUP` operator of the `GROUP BY` clause, the operation results in $n + 1 = 2 + 1 = 3$ groupings.
- Rows based on the values of the first n expressions are called rows or regular rows and the others are called superaggregate rows.

CUBE Operator

```
SELECT        [column,] group_function(column)...
FROM          table
[WHERE        condition]
[GROUP BY     [CUBE] group_by_expression]
[HAVING       having_expression]
[ORDER BY     column];
```

- **CUBE is an extension to the GROUP BY clause.**
- **You can use the CUBE operator to produce cross-tabulation values with a single SELECT statement.**

ORACLE

3-9 Copyright © Oracle Corporation, 2001. All rights reserved.

The CUBE Operator

The CUBE operator is an additional switch in the GROUP BY clause in a SELECT statement. The CUBE operator can be applied to all aggregate functions, including AVG, SUM, MAX, MIN, and COUNT. It is used to produce results sets that are typically used for cross-tabular reports. While ROLLUP produces only a fraction of possible subtotal combinations, CUBE produces subtotals for all possible combinations of groupings specified in the GROUP BY clause, and a grand total.

The CUBE operator is used with an aggregate function to generate additional rows in a results set. Columns included in the GROUP BY clause are cross-referenced to produce a superset of groups. The aggregate function specified in the select list is applied to these groups to produce summary values for the additional superaggregate rows. The number of extra groups in the results set is determined by the number of columns included in the GROUP BY clause.

In fact, every possible combination of the columns or expressions in the GROUP BY clause is used to produce superaggregates. If you have n columns or expressions in the GROUP BY clause, there will be 2^n possible superaggregate combinations. Mathematically, these combinations form an n-dimensional cube, which is how the operator got its name.

By using application or programming tools, these superaggregate values can then be fed into charts and graphs that convey results and relationships visually and effectively.

CUBE Operator: Example

```
SELECT   department_id, job_id, SUM(salary)
FROM     employees
WHERE    department_id < 60
GROUP BY CUBE (department_id, job_id) ;
```

DEPARTMENT_ID	JOB_ID	SUM(SALARY)
10	AD_ASST	4400
10		4400
20	MK_MAN	13000
20	MK_REP	6000
20		19000
50	ST_CLERK	11700
50	ST_MAN	5800
50		17500
	AD_ASST	4400
	MK_MAN	13000
	MK_REP	6000
	ST_CLERK	11700
	ST_MAN	5800
		40900

14 rows selected.

1 2 3 4

ORACLE

3-10 Copyright © Oracle Corporation, 2001. All rights reserved.

Example of a CUBE Operator

The output of the SELECT statement in the example can be interpreted as follows:

- The total salary for every job within a department (for those departments whose department ID is less than 60) is displayed by the GROUP BY clause (labeled 1)
- The total salary for those departments whose department ID is less than 60 (labeled 2)
- The total salary for every job irrespective of the department (labeled 3)
- Total salary for those departments whose department ID is less than 60, irrespective of the job titles (labeled 4)

In the preceding example, all rows indicated as 1 are regular rows, all rows indicated as 2 and 4 are superaggregate rows, and all rows indicated as 3 are cross-tabulation values.

The CUBE operator has also performed the ROLLUP operation to display the subtotals for those departments whose department ID is less than 60 and the total salary for those departments whose department ID is less than 60, irrespective of the job titles. Additionally, the CUBE operator displays the total salary for every job irrespective of the department.

Note: Similar to the ROLLUP operator, producing subtotals in *n* dimensions (that is, *n* columns in the GROUP BY clause) without a CUBE operator requires 2^n SELECT statements to be linked with UNION ALL. Thus, a report with three dimensions requires $2^3 = 8$ SELECT statements to be linked with UNION ALL.

GROUPING Function

```
SELECT     [column,] group_function(column) .. ,
           GROUPING(expr)
FROM       table
[WHERE     condition]
[GROUP BY  [ROLLUP][CUBE] group_by_expression]
[HAVING    having_expression]
[ORDER BY  column];
```

- **The GROUPING function can be used with either the CUBE or ROLLUP operator.**
- **Using the GROUPING function, you can find the groups forming the subtotal in a row.**
- **Using the GROUPING function, you can differentiate stored NULL values from NULL values created by ROLLUP or CUBE.**
- **The GROUPING function returns 0 or 1.**

ORACLE

3-11 Copyright © Oracle Corporation, 2001. All rights reserved.

The GROUPING Function

The GROUPING function can be used with either the CUBE or ROLLUP operator to help you understand how a summary value has been obtained.

The GROUPING function uses a single column as its argument. The *expr* in the GROUPING function must match one of the expressions in the GROUP BY clause. The function returns a value of 0 or 1.

The values returned by the GROUPING function are useful to:

- Determine the level of aggregation of a given subtotal; that is, the group or groups on which the subtotal is based
- Identify whether a NULL value in the expression column of a row of the result set indicates:
  - A NULL value from the base table (stored NULL value)
  - A NULL value created by ROLLUP/CUBE (as a result of a group function on that expression)

A value of 0 returned by the GROUPING function based on an expression indicates one of the following:

- The expression has been used to calculate the aggregate value.
- The NULL value in the expression column is a stored NULL value.

A value of 1 returned by the GROUPING function based on an expression indicates one of the following:

- The expression has not been used to calculate the aggregate value.
- The NULL value in the expression column is created by ROLLUP or CUBE as a result of grouping.

GROUPING Function: Example

```
SELECT   department_id DEPTID, job_id JOB,
         SUM(salary),
         GROUPING(department_id) GRP_DEPT,
         GROUPING(job_id) GRP_JOB
FROM     employees
WHERE    department_id < 50
GROUP BY ROLLUP(department_id, job_id);
```

DEPTID	JOB	SUM(SALARY)	GRP_DEPT	GRP_JOB
10	AD_ASST	4400	0	0
10		4400	0	1
20	MK_MAN	13000	0	0
20	MK_REP	6000	0	0
20		19000	0	1
		23400	1	1

6 rows selected.

1 2 3

ORACLE

3-12 Copyright © Oracle Corporation, 2001. All rights reserved.

Example of a GROUPING Function

In the example in the slide, consider the summary value 4400 in the first row (labeled 1). This summary value is the total salary for the job ID of AD_ASST within department 10. To calculate this summary value, both the columns DEPARTMENT_ID and JOB_ID have been taken into account. Thus a value of 0 is returned for both the expressions GROUPING(department_id) and GROUPING(job_id).

Consider the summary value 4400 in the second row (labeled 2). This value is the total salary for department 10 and has been calculated by taking into account the column DEPARTMENT_ID; thus a value of 0 has been returned by GROUPING(department_id). Because the column JOB_ID has not been taken into account to calculate this value, a value of 1 has been returned for GROUPING(job_id). You can observe similar output in the fifth row.

In the last row, consider the summary value 23400 (labeled 3). This is the total salary for those departments whose department ID is less than 50 and all job titles. To calculate this summary value, neither of the columns DEPARTMENT_ID and JOB_ID have been taken into account. Thus a value of 1 is returned for both the expressions GROUPING(department_id) and GROUPING(job_id).